

ASPERITAS AI AGENT FACTORY PLAYBOOK

Vibe Coding + ChatGPT + Codex + GitHub + Skills + Workflow Pipeline

신규 개발자 온보딩 / 독자적 에이전트 개발 / 오픈소스 흡수 / 검증 게이트 / 커밋·PR 운영 / 성능향상 루프

Version: 2.0

Date: 2026-06-15

Owner: COO & AI Manager Office

Scope: Asperitas Inc. internal agent-development operating manual

핵심 명령

모든 독자적 에이전트 개발은 “초안 -> 프롬프트 -> 구현 -> 검증 -> 확인 -> GitHub 커밋/PR -> 회고 -> 스킬/워크플로우 업데이트” 루프로 운영한다. AI가 만든 코드는 믿는 것이 아니라 검증하고, 검증된 절차는 다시 AGENTS.md와 Skills로 제품화한다.

Revision Log - v2.0 업데이트 내용

버전	날짜	핵심 변경	의미
v1.0	2026-06-14	기본 개발 루프, GitHub, Codex 프롬프트, 온보딩 골격 정리	초기 플레이북
v2.0	2026-06-15	Agent Factory 운영체제로 재편. 신규 개발자 루프, 오픈소스 흡수 프로토콜, Skill/Workflow 확장 루프, 검증 게이트, GitHub governance, 복붙 프롬프트 확장	실제 개발 표준 문서

v2.0의 핵심 차이

v1.0 이 “어떻게 하면 되는지”를 설명한 문서라면, v2.0 은 신규 개발자가 합류했을 때 그대로 따라 하는 운영 매뉴얼이다. Codex 에게 시키는 법, GitHub 에 남기는 법, 실패를 복구하는 법, 다음 에이전트에도 반복 가능한 스킬로 바꾸는 법까지 포함한다.

문서 사용 규칙

- 신규 개발자는 1~3 장을 먼저 읽고, 4 장의 표준 개발 루프를 그대로 따른다.
- 기존 개발자는 5~8 장을 이용해 Codex 프롬프트, 검증 게이트, GitHub PR 단위를 정한다.
- AI/플랫폼 담당자는 9~12 장을 이용해 AGENTS.md, Skills, Workflow Pipeline, Subagent 체계를 개선한다.
- 오픈소스를 가져올 때는 7 장을 반드시 통과한다. 라이선스와 보안 검토 없는 복붙은 금지한다.
- 각 MVP 완료 후 13 장의 Decision Log 형식으로 판단 근거와 남은 리스크를 남긴다.

Contents

1. 0. Executive Command - 왜 이 문서가 필요한가
2. 1. Asperitas Agent Factory 의 큰 그림
3. 2. 현재 개발 방식의 표준화: 초안 -> 프롬프트 -> 검증 -> 확인 -> 커밋
4. 3. 역할 분담: ChatGPT, Codex, Claude Code, GitHub, 사람
5. 4. 신규 에이전트 개발 Step-by-Step
6. 5. Codex Prompt Operating System
7. 6. GitHub 연동, 브랜치, PR, 커밋, 보호 규칙
8. 7. 오픈소스 흡수 프로토콜: 좋은 것 다 굶되 안전하게
9. 8. 검증 게이트: pytest, artifact verify, eval, diff, review
10. 9. AGENTS.md, Skills, Rules, Workflows, Pipeline 구축
11. 10. 새 채팅에서 성능향상/스킬추가/워크플로우 구축하는 법
12. 11. Claude Code/Skills/Subagents 와 Codex 병행 전략
13. 12. 신규 개발자 7 일 온보딩 플랜
14. 13. 복붙용 프롬프트 라이브러리
15. 14. 운영 리스크와 금지 사항
16. 15. KPI 와 성능향상 루프
17. 16. Appendix: 파일 템플릿, 체크리스트, 공식 출처

0. Executive Command - 왜 이 문서가 필요한가

Asperitas가 앞으로 개발할 독자적 에이전트는 하나가 아니다. RAG Agent, Literature Intelligence Agent, Compliance Agent, Protocol Design Agent, Investor/IR Agent, Biofoundry Automation Agent, Market Intelligence Agent 등으로 확장될 수 있다. 따라서 매번 감으로 시작하면 개발 속도는 빨라 보이지만, 결과는 재현되지 않는다.

이 문서의 목적은 “AI를 잘 쓰는 개인기”를 “회사가 반복 가능한 개발 공정”으로 바꾸는 것이다. 초안, 프롬프트, 구현, 검증, GitHub 커밋, 회고, 스킬 업데이트를 하나의 공장 라인처럼 고정한다.

CEO-grade 지시	Asperitas 식 번역	현장 실행
AI를 장난감처럼 쓰지 말고 생산라인처럼 써라.	모든 AI 개발 요청은 루프와 게이트를 가져야 한다.	Task Brief, Constraints, Tests, Report를 프롬프트에 포함한다.
속도와 품질을 동시에 가져가라.	빠르게 만들어, 검증 없이 main에 넣지 않는다.	branch -> test -> verify -> PR -> review -> merge.
실패를 숨기지 말고 자산화하라.	오류는 Failure Taxonomy와 Decision Log로 남긴다.	반복 오류는 AGENTS.md/Skills에 반영한다.
좋은 외부 자산은 흡수하라. 단, 법적/보안 오염은 금지다.	오픈소스는 Scout -> License -> Security -> Benchmark -> Adapt 순서로만 흡수한다.	NOTICE, LICENSE, SBOM, dependency review를 남긴다.
사람은 기준을 세우고, AI는 반복을 압축한다.	최종 승인권은 사람에게 있다.	AI output을 evidence로 쓰지 말고 검증 대상으로 둔다.

Bottom Line

vibe coding은 “대충 느낌대로 코딩”이 아니다. Asperitas 기준에서 vibe coding은 인간이 큰 방향과 판단 기준을 빠르게 제시하고, AI가 코드·테스트·문서·리팩터링 반복을 압축하며, GitHub와 검증 게이트가 결과를 통제하는 운영 방식이다.

0.1 이 문서가 신규 개발자에게 요구하는 태도

- 코드보다 먼저 목적을 이해한다. 지금 만드는 기능이 어떤 MVP 단계, 어떤 에이전트, 어떤 평가 지표에 연결되는지 확인한다.
- AI가 작성한 코드도 본인이 작성한 코드처럼 책임진다. “Codex가 그렇게 했다”는 면책 사유가 아니다.
- 작은 변경을 선호한다. 한 번에 구조 전체를 뒤엎기보다, 테스트 가능한 최소 단위로 나눈다.
- 검증 결과를 캡처하거나 로그로 남긴다. 나중에 다른 사람이 이어받을 수 있어야 한다.
- 반복되는 프롬프트는 개인 메모가 아니라 repo의 AGENTS.md 또는 .agents/skills로 승격한다.

1. Asperitas Agent Factory의 큰 그림

Agent Factory는 하나의 에이전트를 만드는 프로젝트가 아니라, 여러 독자적 에이전트를 반복적으로 생산하기 위한 운영체제다. 핵심 구성은 Agent Brief, Codex Prompt, Implementation Branch, Verification Gate, GitHub Memory, Skill Upgrade Loop로 나뉜다.

Agent Factory Loop

```

[Idea / Business Need]
↓
[Agent Brief: objective, users, data, risks, non-goals]
↓
[Prompt Engineering: ChatGPT converts rough idea into Codex task]
↓
[Codex Implementation: smallest safe change + tests]
↓
[Verification Gate: pytest + artifact verify + eval + diff]
↓
[Human Review: screenshot/log/result check]
↓
[GitHub Commit/PR: traceable organization memory]
↓
[Retrospective: update AGENTS.md / Skills / workflow templates]
↓
[Next Agent / Next MVP]
  
```

레이어	역할	대표 산출물
Strategy Layer	왜 만드는지, 어느 MVP/사업목표에 연결되는지 정의	Agent Brief, Phase Roadmap, success metric
Instruction Layer	AI가 따라야 할 지속 규칙을 저장	AGENTS.md, CLAUDE.md, .agents/skills, .claude/rules
Implementation Layer	Codex/Claude가 코드·테스트·문서 변경 수행	feature branch, commits, test files
Verification Layer	기능과 품질을 객관적으로 확인	pytest logs, eval metrics, artifact verification, CI
Governance Layer	보안, 라이선스, 브랜치 보호, 승인 절차 관리	PR review, license ledger, branch rules, CODEOWNERS
Learning Layer	반복 실수와 성공 패턴을 재사용 가능한 자산으로 축적	Decision Log, Failure Taxonomy, Skills update

1.1 Agent Factory의 5대 원칙

- Scope Lock: 무엇을 바꿀지보다 무엇을 바꾸지 않을지를 먼저 정한다.
- Small Safe Change: 한 번의 작업은 한 가지 목표만 달성한다. 대형 리팩터링과 기능 추가를 섞지 않는다.
- Verification First: 구현 성공의 기준은 “AI가 완료했다”가 아니라 테스트, 평가, diff, 로그가 통과했다는 것이다.
- GitHub as Memory: GitHub는 백업이 아니라 회사의 기술 기억장치다. PR, issue, commit, decision log가 조직 학습 데이터가 된다.
- Skill Compounding: 반복되는 지시, 검증, 리뷰 기준은 AGENTS.md와 Skills로 축적해 다음 작업의 품질을 높인다.

1.2 에이전트 유형별 개발 우선순위

우선순위	에이전트	목적	초기 MVP 산출물
P0	Asperitas RAG Agent	회사 내부 문서, 규제, 논문, 산업자료를 근거 기반으로 검색/답변	ingest -> chunk -> retrieval eval -> source citation
P1	Literature Intelligence Agent	논문 탐색, 방법론 요약, 실험 설계 근거 추출	paper metadata parser, claim/evidence table
P1	Compliance & Biosafety Agent	규제/윤리/생물안전 체크리스트 자동화	risk taxonomy, SOP checklist, audit log
P2	Protocol Design Agent	DBTL 실험 프로토콜 초안과 변수 설계 지원	protocol template, parameter extraction
P2	Investor/IR Agent	IR 문서, 시장 벤치마크, 투자자 질의 응답 지원	Q&A bank, competitor matrix
P3	Biofoundry Automation Agent	LIMS/로봇/스케줄링과 연결되는 자동화 레이어	API mock, workflow scheduler spec

2. 현재 개발 방식의 표준화: 초안 -> 프롬프트 -> 검증 -> 확인 -> 커밋

현재 사용자가 수행 중인 개발 루프는 그대로 유지하되, 신규 개발자가 놓치지 않도록 체크포인트를 명확히 한다. 이 루프는 모든 MVP와 모든 독자적 에이전트에 적용된다.

단계	무엇을 하는가	누가 주도하는가	완료 조건
1. 초안	비즈니스/기술 목적을 거칠게 말한다.	사용자/PM	목표, 현재 상태, 원하는 변화가 적혀 있음
2. 프롬프트화	ChatGPT가 Codex용 실행 지시로 변환한다.	ChatGPT	Task, Context, Constraints, Verification, Report 포함
3. Preflight	Codex가 repo 상태, 관련 파일, 위험을 읽고 계획한다.	Codex	수정 전 파일 목록, 예상 변경, 테스트 계획 보고
4. 구현	최소 변경으로 코드/테스트/문서 수정.	Codex	변경 파일이 scope 안에 있음
5. 검증	pytest, eval, artifact verification, lint, git diff 확인.	Codex + 개발자	모든 필수 체크 통과 또는 실패 원인 명시
6. 확인	사용자가 결과 화면/로그/요약을 보고 승인 여부 결정.	사용자/리드	버그/불확실성/남은 리스크 확인
7. 커밋/푸시	GitHub Desktop 또는 CLI로 commit/push/PR 생성.	개발자	commit message, PR summary, tests run 기록
8. 회고	반복 오류와 새 기준을 AGENTS.md/Skills로 승격.	AI Manager	다음 작업 품질을 높이는 durable rule 추가

2.1 이 루프에서 절대 생략하면 안 되는 것

- 현재 branch와 working tree 상태 확인: `git status`, `git branch --show-current`, `git log -1 --oneline`.
- 변경 전 관련 파일 읽기: 구현 전에 구조를 모르면 먼저 inspect 한다.
- 테스트 추가 또는 기존 테스트 업데이트: behavior가 바뀌면 반드시 테스트도 바뀐다.
- 검증 명령 실행: pytest, artifact verify, retrieval eval 등 해당 작업에 맞는 체크를 실행한다.
- git diff 검토: AI가 scope 밖 파일을 건드리지 않았는지 확인한다.
- 보고서 작성: objective, files changed, tests run, metrics, risks를 남긴다.

2.2 MVP 별 루프 적용 예시

MVP 유형	Codex 작업 예시	필수 검증
Chunking/Retrieval	chunk schema, heading metadata, retrieval scoring 수정	pytest + retrieval eval before/after + sample chunk inspect
Ingestion	PDF/MD/HTML parser, metadata extraction 추가	fixture ingest + artifact verify + license metadata check
Evaluation	golden query, metrics, failure taxonomy 추가	eval snapshot + regression threshold + report
UI/CLI	명령어 또는 간단한 인터페이스 추가	CLI smoke test + docs update + backward compatibility
Compliance	risk label, audit log, policy checker 추가	unit tests + red-team sample + false positive review

3. 역할 분담: ChatGPT, Codex, Claude Code, GitHub, 사람

AI 도구를 많이 쓰는 것보다 중요한 것은 역할을 분리하는 것이다. 한 도구에게 모든 것을 맡기면 context rot, 과잉 수정, 검증 누락이 발생한다. Asperitas 식 운영에서는 ChatGPT를 설계/프롬프트/검증 파트너로, Codex를 구현/테스트 실행 엔진으로, GitHub를 기억장치와 품질 게이트로 둔다.

주체	가장 잘하는 일	맡기면 안 되는 일	대표 산출물
User / COO AI Manager	목표 설정, 우선순위, 최종 승인, 리스크 판단	코드 변경을 눈으로만 승인하고 검증 생략	MVP objective, approval, roadmap
ChatGPT	초안 정리, 프롬프트 작성, 아키텍처 검토, 리스크 반론, 보고서 해석	repo 내부 파일을 실제로 수정했다고 가정	Codex prompt, review checklist, decision memo
Codex	repo 읽기, 코드 수정, 테스트 작성/실행, diff 요약	비즈니스 우선순위 최종 판단, 라이선스 독단 승인	commit-ready changes, test report
Claude Code	대규모 코드 탐색, skills/subagents/hooks 기반 워크플로우, 대안 구현 검토	Codex와 같은 branch에서 동시 수정해 충돌 만들기	review notes, second-opinion patch, skill
GitHub	branch, PR, commit, review, CI, protected main, issue 관리	설명 없는 dump 저장소로 사용	PR, CI logs, release notes

3.1 언제 ChatGPT를 쓰는가

- 아이디어가 아직 흐릿할 때: Agent Brief와 MVP scope로 바꾼다.
- Codex에게 복붙할 프롬프트가 필요할 때: Task/Context/Constraints/Report 구조로 만든다.
- Codex 결과가 맞는지 해석이 필요할 때: 테스트 로그, diff 요약, 리스크를 검토한다.
- 새 채팅에서 성능향상/스킬추가/워크플로우 구축을 설계할 때: durable rule과 skill template으로 정리한다.
- 개발자가 온보딩될 때: 전체 프로세스를 교육 자료와 체크리스트로 만든다.

3.2 언제 Codex를 쓰는가

- 실제 repo에서 파일을 읽고 수정해야 할 때.
- 테스트를 만들고 실행해야 할 때.
- git diff를 보고 변경 범위를 요약해야 할 때.
- 오래 걸리는 구현/리팩터링/마이그레이션을 작은 단계로 처리해야 할 때.
- AGENTS.md와 repo-specific skill을 따라 일관된 방식으로 작업해야 할 때.

3.3 언제 Claude Code를 병행하는가

- Codex 결과를 독립적으로 리뷰하거나 다른 구현 전략을 비교할 때.
- skills, hooks, subagents를 이용해 반복 작업을 도구화할 때.
- 긴 문맥의 코드베이스 이해, 아키텍처 탐색, 리팩터링 계획이 필요할 때.
- 단, 같은 파일을 동시에 수정하지 않는다. 한 도구가 patch를 만들고, 다른 도구는 review 또는 follow-up branch에서 검토한다.

4. 신규 에이전트 개발 Step-by-Step

아래 과정은 앞으로 어떤 독자적 에이전트를 만들더라도 그대로 적용한다. 신규 개발자는 이 장을 순서대로 따라가면 된다.

Step 0. Agent Brief 작성

질문	작성 예시
이 에이전트는 누구의 어떤 병목을 줄이는가?	R&D 팀이 논문/규제/내부 문서를 찾는 시간을 줄인다.
입력은 무엇인가?	질문, 문서 파일, URL, 데이터베이스 레코드.
출력은 무엇인가?	근거 인용 답변, source table, risk label, action item.
절대 하면 안 되는 일은?	근거 없는 실험 조건 추천, 규제 단정, 민감 정보 외부 전송.
성공 기준은?	golden query Top-K recall, citation accuracy, zero crash, eval threshold.

Agent Brief Template

```
# AGENT_BRIEF.md template
Agent name:
Owner:
MVP stage:
Primary user:
Business objective:
Input data:
Output contract:
Non-goals:
Safety/compliance constraints:
Evaluation metric:
Required tests:
Expected files:
Open questions:
```

Step 1. Repo 준비

23. GitHub repository 를 만든다. 내부/비공개가 기본이다. 공개 여부는 라이선스, IP, 데이터 민감도 검토 후 결정한다.
24. README.md, LICENSE, .gitignore, AGENTS.md, tests/, scripts/, src/ 기본 구조를 만든다.
25. GitHub Desktop 또는 CLI 로 local clone 을 만든다.
26. main branch 는 protected branch 로 운영하고, 직접 commit 은 예외로 둔다.
27. 초기 CI 는 최소한 pytest 를 돌리도록 설정한다.

```
# 권장 초기 구조
repo/
├── AGENTS.md
├── README.md
├── LICENSE
├── pyproject.toml
├── src/<package_name>/
├── tests/
├── scripts/
├── data/ # raw data 는 원칙적으로 git 에 넣지 않음
├── docs/
├── .agents/skills/
├── .github/workflows/
└── 09_LOGS/decision_logs/
```

Step 2. ChatGPT 로 Codex 프롬프트 생성

사용자는 자연어로 말해도 된다. ChatGPT 는 이를 Codex 가 실행 가능한 구조로 바꿔야 한다. Asperitas 기본 프롬프트 구조는 아래와 같다.

Codex Base Prompt

```
Use AGENTS.md and the relevant .agents/skills instructions.

Task:
[구현할 작업을 한 문장으로]

Context:
- Current repo: [repo name]
- Current MVP stage: [MVP-xxx]
- Target files: [known files or "inspect first"]
- Current behavior: [observed behavior]
- Desired behavior: [acceptance criteria]

Constraints:
- Do not delete files.
- Make the smallest safe change.
- Preserve backward compatibility.
- Add or update tests.
- Run pytest.
```


- Run artifact verification.
- Run retrieval eval if retrieval/chunking/scoring is affected.
- Update docs if behavior changes.
- Do not commit unless explicitly asked.

Report:

1. objective
2. files changed
3. tests run
4. eval metrics before/after
5. risks / follow-ups

Step 3. Codex Preflight

- Codex 에게 바로 구현하지 말고 먼저 repo 상태와 관련 파일을 inspect 하게 한다.
- 현재 branch, HEAD, working tree clean 여부를 확인하게 한다.
- 어떤 파일을 바꿀지, 어떤 테스트를 돌릴지, 위험이 무엇인지 보고하게 한다.
- 복잡하면 Plan mode 또는 “plan first, do not edit yet”를 요구한다.

```
Preflight only. Do not edit files yet.
Inspect the repository and report:
1. current branch / HEAD / git status
2. files relevant to this task
3. proposed smallest safe change
4. tests and verification commands to run
5. risks, unknowns, and whether this should be split into smaller tasks
```

Step 4. 구현

- Preflight 가 합리적이면 Codex 에게 구현을 승인한다.
- MVP 단위에서는 “작은 변경 + 테스트 + 문서”를 기본 패키지로 본다.
- 대형 리팩터링, 성능개선, 기능추가를 한 PR 에 섞지 않는다.
- 중간에 예상보다 파일 변경이 커지면 멈추고 다시 scope 를 줄인다.

Step 5. 검증

```
# Python repo 기본 검증 예시
python -m pytest
python scripts/verify_artifacts.py
python scripts/run_retrieval_eval.py # retrieval/chunking/scoring affected only
git diff --stat
git diff -- src tests scripts docs
```

검증 유형	언제 필요한가	통과 기준
Unit test	함수/스키마/로직 변경	pytest pass
Artifact verification	data/chunks.jsonl, report, generated artifact 영향	schema valid, no missing mandatory fields
Retrieval eval	retrieval/chunking/scoring 변경	baseline 대비 악화 없음 또는 명시적 개선
Manual sample check	문서/검색/출력 품질 판단 필요	샘플 3~5 개가 기대 행동과 일치
Git diff review	항상	scope 밖 변경 없음, 삭제/대형 포매팅 없음

Step 6. 확인과 승인

개발자가 Codex 보고서를 사용자/리드에게 전달할 때는 화면 캡처보다 텍스트 요약이 더 중요하다. 캡처는 보조 증거이고, 최종 판단은 파일 변경, 테스트 결과, eval 지표, 남은 리스크로 한다.

```
MVP Completion Report
1. Objective:
2. Files changed:
3. Behavior changed:
4. Tests run:
5. Eval metrics before/after:
6. Git status:
7. Risks / follow-ups:
8. Recommended next MVP:
```

Step 7. GitHub 커밋/푸시/PR

- 변경이 작고 검증을 통과하면 commit 한다.
- commit message 는 무엇을 바꿨는지보다 왜 바꿨는지를 담는다.
- main 에 직접 push 하지 않고 feature branch + PR 을 기본으로 한다.
- PR 본문에는 objective, files changed, tests run, risks 를 포함한다.

- merge 후에는 GitHub Desktop에서 local main을 pull하여 최신 상태로 동기화한다.

```
# Commit message examples
feat(retrieval): add section-aware chunk metadata
fix(eval): preserve existing pass/fail behavior in taxonomy report
test(chunking): add markdown heading fixtures
docs(mvp): document MVP-011 failure taxonomy preflight
```

5. Codex Prompt Operating System

Codex 는 단순 코드 생성기가 아니라 repo-aware coding agent 로 다뤄야 한다. 공식 Codex 문서의 핵심 패턴은 목표, 컨텍스트, 제약, 완료 조건을 명확히 주고, 어려운 작업은 계획부터 세우며, 반복 지시는 AGENTS.md 로 재사용하라는 것이다. v2.0 에서는 이를 Asperitas 식 Prompt OS 로 고정한다.

5.1 좋은 Codex 프롬프트의 8 요소

요소	설명	나쁜 예	좋은 예
Goal	무엇을 달성할지	검색 좋아지게 해줘	retrieval eval 에서 Top-3 evidence recall 을 개선하되 기존 PASS/FAIL behavior 는 보존
Context	관련 파일/로그/상태	알아서 찾아	src/retrieval_mvp003.py, tests/test_retrieval_eval.py, data/chunks.jsonl inspect first
Constraints	금지/제약	잘 해줘	Do not delete files, preserve schema backward compatibility
Acceptance	완료 기준	완성되면 알려줘	pytest pass, artifact verify pass, eval before/after reported
Scope	변경 범위	전체 최적화	Only adjust scoring; do not change ingestion or chunking
Safety	보안/컴플라이언스	문제 없게	No external network, no secrets, no raw sensitive data in commits
Report	보고 형식	요약해줘	objective/files/tests/metrics/risks
Stop rule	멈춰야 할 조건	막히면 알아서	If more than 5 files need changes, stop and propose split

5.2 난이도별 프롬프트 모드

작업 난이도	Codex 모드	지시 방식	예시
Low	Direct patch	바로 수정 가능	오타, 작은 테스트 추가, docs typo
Medium	Plan + implement	짧은 계획 후 수정	스키마 필드 추가, CLI 옵션 추가
High	Preflight only -> approve -> implement	수정 전 반드시 계획 검토	retrieval scoring 변경, chunker 변경
Extra High	Split into milestones	여러 PR 로 분할	multi-agent pipeline, DB schema migration

5.3 Codex 에게 항상 붙이는 Asperitas 안전문

Asperitas safety constraints:

- Treat this as production-adjacent internal code.
- Do not remove backward compatibility unless explicitly instructed.
- Do not delete files or rewrite unrelated files.
- Do not ingest external data or call external services unless explicitly approved.
- Never commit secrets, credentials, private keys, or raw sensitive company data.
- If the task touches compliance, biosafety, legal, licensing, or IP, flag it in the report.
- Prefer small, reviewable diffs over large rewrites.
- If the requirement is ambiguous, make the smallest reasonable assumption and state it.

5.4 Codex 결과 리뷰 프롬프트

Codex 가 작업을 끝낸 뒤, 같은 Codex 또는 다른 AI 에게 리뷰만 시킬 수 있다. 이때는 수정 금지로 시작한다.

Review only. Do not edit files.
Inspect the current git diff and verify whether the implementation satisfies the task.
Check:

1. scope creep
2. backward compatibility
3. missing tests
4. brittle logic or hidden assumptions
5. security/license/compliance risk
6. whether docs need updates

Return a blocking/non-blocking issue list and recommended next action.

6. GitHub 연동, 브랜치, PR, 커밋, 보호 규칙

GitHub는 개발 협업 도구를 넘어 Agent Factory의 기억장치다. Codex가 만든 결과, 사람이 검토한 근거, 테스트 로그, PR 토론, 릴리스 노트가 모두 누적되어 다음 AI 작업의 컨텍스트가 된다.

6.1 GitHub 기본 세팅

28. Repository visibility: 회사 핵심 코드는 Private 기본. Public 전환은 라이선스/IP/보안 검토 후 결정.
29. Collaborators: 필요한 개발자만 초대. 최소 권한 원칙 적용.
30. Branch strategy: main은 안정판, feature/*는 작업 브랜치, release/*는 배포 후보.
31. Branch protection: main에 직접 force push/delete 금지. PR review와 status checks 요구.
32. CI: 최소 pytest. 이후 lint, type check, security scan, artifact verify 추가.
33. Issue/PR templates: objective, checklist, tests, risks를 템플릿화.
34. CODEOWNERS: 핵심 영역별 승인자를 지정. 예: retrieval, ingestion, compliance, infra.

6.2 브랜치 전략

브랜치	용도	규칙
main	항상 안정적인 기준선	직접 커밋 금지, PR merge만 허용
feature/mvp-xxx-short-name	기능 개발	작은 scope, 테스트 포함
fix/bug-short-name	버그 수정	재현 테스트 먼저 추가
docs/topic	문서 수정	코드 변경과 섞지 않기
experiment/topic	실험/벤치마크	main merge 전 cleanup 필요

6.3 GitHub Desktop 기준 커밋 절차

35. 좌측 변경 파일 목록을 본다. 예상하지 못한 파일이 있으면 커밋 전 중단한다.
36. 각 파일 diff를 열어 scope 밖 변경, 대량 포매팅, 삭제가 없는지 확인한다.
37. Summary에 conventional commit 형식으로 제목을 쓴다.
38. Description에 tests run과 risk를 짧게 쓴다.
39. Commit to branch를 누른다.
40. Push origin을 누른다.
41. GitHub 웹에서 PR을 생성하고 템플릿을 채운다.
42. CI 통과와 review 후 merge한다.

6.4 PR 템플릿

```
## Objective

## What changed
-

## Verification
- [ ] pytest
- [ ] artifact verification
- [ ] retrieval eval before/after, if applicable
- [ ] manual sample check, if applicable

## Risk review
- Backward compatibility:
- Data/security:
- License/IP:
- Biosafety/compliance:

## Follow-ups
-
```

6.5 Protected main 운영 원칙

- main branch는 항상 재현 가능한 상태여야 한다.
- 필수 status check는 이름이 중복되지 않게 관리한다.
- PR은 Files changed와 Checks 탭을 모두 확인한다.
- 실패한 CI를 무시하고 merge하지 않는다.
- 긴급 수정도 hotfix branch를 통해 최소 테스트 후 merge한다.

7. 오픈소스 흡수 프로토콜: 좋은 것 다 굶되 안전하게

“좋은 GitHub 오픈소스를 다 굶어오기”는 방향 자체는 맞다. 다만 회사 코드베이스에서는 무작정 복붙하면 라이선스, 보안, 유지보수, 아키텍처 오염이 발생한다. 따라서 Asperitas는 오픈소스를 코드 원재료가 아니라 검증 대상 자산으로 다룬다.

7.1 6 단계 흡수 루프

단계	목적	실행 체크
1. Scout	좋은 후보 찾기	GitHub, 공식 문서, 논문 코드, benchmark repo 탐색
2. License	법적 사용 가능성 확인	LICENSE, NOTICE, dependency license, copyleft 여부
3. Security	보안 위험 확인	secret, malware, dependency vulnerability, abandoned repo
4. Benchmark	실제로 좋은지 검증	sample data, performance, tests, maintainability
5. Adapt	그대로 복붙하지 않고 회사 구조에 맞게 흡수	interface wrapper, minimal module, test fixture
6. Ledger	무엇을 왜 가져왔는지 기록	OPEN_SOURCE_LEDGER.md, citation, version, commit hash

7.2 금지되는 오픈소스 사용 방식

- LICENSE 확인 없이 소스 파일을 복사하는 것.
- StackOverflow/GitHub/Gist 코드를 출처 없이 핵심 로직으로 넣는 것.
- 보안 스캔 없이 오래된 dependency를 추가하는 것.
- 실험 repo 코드를 프로덕션 경로에 바로 넣는 것.
- 회사 내부 데이터와 외부 오픈소스 예제 코드를 섞어 공개 repo에 올리는 것.
- AI가 추천한 repo를 실제 확인 없이 “유명하다”고 가정하는 것.

7.3 우선 탐색할 오픈소스 카테고리

카테고리	왜 보는가	흡수 방식
Coding agents / CLI	Codex/Claude 방식의 repo-aware workflow 이해	직접 복붙보다 architecture와 prompt pattern 학습
RAG frameworks	chunking, retrieval, eval, citation pipeline 참고	핵심 컨셉만 adapter로 흡수
Evaluation frameworks	golden dataset, metrics, regression testing 참고	eval script와 report format 참고
Bioinformatics tools	sequence parsing, annotation, database handling 참고	라이선스 확인 후 wrapper로 연결
Workflow orchestration	pipeline, DAG, async job, retries 참고	작은 job runner부터 도입
Security/license scanners	dependency, license, SBOM 관리	CI에 별도 check로 추가

7.4 OPEN_SOURCE_LEDGER.md 템플릿

```
# Open Source Ledger

## [Library or repo name]
- URL:
- Version / commit hash:
- License:
- Date reviewed:
- Reviewer:
- Purpose:
- Files or concepts adopted:
- Direct code copied? yes/no
- Modifications made:
- Security review:
- Dependency impact:
- Replacement plan if abandoned:
- Notes:
```

7.5 오픈소스 후보를 Codex에게 조사시키는 프롬프트

```
Research only. Do not modify repository files.
Find open-source projects relevant to [topic]. For each candidate, report:
1. repository URL
2. license
3. activity/maintenance signal
4. architecture idea worth learning
5. whether direct code adoption is legally/technically risky
6. how to adapt the idea without copying large code blocks
7. recommended next action: reject / study / prototype / adopt
Do not add dependencies yet.
```


8. 검증 게이트: pytest, artifact verify, eval, diff, review

Agent Factory 에서 검증은 옵션이 아니라 제품 생산 라인의 게이트다. Codex 가 “완료”라고 말해도, 검증 게이트를 통과하지 못하면 완료가 아니다.

Gate	목적	실행 명령/산출물	실패 시 처리
G0 Scope Gate	작업 범위가 작고 안전한지 확인	Preflight report	작업 분할
G1 Unit Gate	기본 로직이 깨지지 않았는지 확인	python -m pytest	실패 테스트부터 수정
G2 Artifact Gate	생성 파일 구조/스키마 확인	scripts/verify_artifacts.py	schema/metadata 수정
G3 Eval Gate	검색/분류/생성 품질 regression 확인	run_retrieval_eval.py, eval report	before/after 비교 후 원인 분석
G4 Diff Gate	예상 밖 변경 탐지	git diff --stat, git diff	scope creep 제거
G5 Review Gate	사람/AI 리뷰로 품질 확인	PR review, review prompt	blocking issue 해결

8.1 Retrieval/Chunking 변경 시 추가 기준

- 청킹 변경은 data/chunks.jsonl 의 sample 을 직접 확인한다.
- 스키마에 필드를 추가하면 기존 필드와 backward compatibility 를 유지한다.
- retrieval scoring 변경은 eval before/after 를 반드시 비교한다.
- PASS/FAIL behavior 가 있는 eval 은 기존 판정 로직을 악화하지 않는다.
- 새 metadata 는 비어 있을 수 있는 경우를 테스트한다.

8.2 실패를 자산화하는 Failure Taxonomy

실패 유형	예시	대응
SPEC_AMBIGUITY	요구사항이 너무 넓은	Agent Brief 와 non-goals 재작성
SCOPE_CREEP	불필요한 파일까지 수정	git diff 로 되돌리고 task split
TEST_GAP	기능은 바뀌었는데 테스트 없음	regression test 추가
EVAL_REGRESSION	retrieval 지표 하락	scoring/chunking 원인 분석
DATA_SCHEMA_BREAK	기존 artifact reader 가 깨짐	compatibility adapter 추가
LICENSE_RISK	오픈소스 출처/라이선스 불명	adoption 중단, ledger 기록
SECURITY_RISK	secret, external call, unsafe dependency	즉시 제거, secret scan

8.3 Codex 검증 보고 프롬프트

```
Run verification for the current changes.
Required:
1. git status
2. git diff --stat
3. pytest
4. artifact verification if scripts/verify_artifacts.py exists
5. retrieval eval if retrieval/chunking/scoring was affected
6. summarize failures with likely root cause
7. do not hide skipped tests or warnings
Report pass/fail clearly.
```

9. AGENTS.md, Skills, Rules, Workflows, Pipeline 구축

반복되는 지시는 매번 채팅에 붙여넣지 않는다. AGENTS.md, .agents/skills, CLAUDE.md, .claude/rules 같은 구조로 승격해야 한다. 이것이 AI 활용 성능을 장기적으로 끌어올리는 핵심이다.

9.1 AGENTS.md 에 들어갈 내용

항목	내용 예시
Repo overview	이 repo 의 목적, 핵심 패키지, MVP 단계
Architecture	src, scripts, tests, data, docs 의 역할
Commands	setup, test, eval, artifact verify 명령
Conventions	naming, schema compatibility, commit style
Safety rules	delete 금지, external call 금지, secret 금지
Definition of done	테스트, eval, docs, report 기준
Reporting format	objective, files, tests, metrics, risks

9.2 AGENTS.md 기본 템플릿

```
# AGENTS.md

## Repository purpose
This repository builds Asperitas internal AI agents. Treat all work as production-adjacent.

## Layout
- src/: core package code
- tests/: unit and regression tests
- scripts/: verification, ingestion, eval utilities
- docs/: design docs and operating manuals
- data/: generated artifacts; do not commit raw sensitive data
- .agents/skills/: reusable task procedures
- 09_LOGS/decision_logs/: decisions and MVP closure reports

## Standard workflow
1. Inspect before editing.
2. Make the smallest safe change.
3. Preserve backward compatibility.
4. Add/update tests.
5. Run pytest and relevant verification scripts.
6. Report objective, files changed, tests, metrics, risks.

## Do not
- Do not delete files unless explicitly approved.
- Do not commit secrets or raw sensitive company data.
- Do not add external dependencies without explaining why.
- Do not weaken existing PASS/FAIL behavior.
```

9.3 Skill 로 승격해야 하는 경우

- 같은 프롬프트를 세 번 이상 반복해서 붙여넣었다.
- 특정 작업이 여러 단계 체크리스트를 요구한다.
- 새 개발자가 매번 실수하는 절차가 있다.
- 코드 리뷰, 검증, 오픈소스 라이선스 점검처럼 독립된 절차가 있다.
- 특정 에이전트/모듈에만 적용되는 규칙이 있다.

9.4 .agents/skills 구조

```
.agents/skills/
├── mvp-implementation/
│   └── SKILL.md
├── retrieval-eval/
│   └── SKILL.md
├── artifact-verification/
│   └── SKILL.md
├── open-source-intake/
│   └── SKILL.md
├── pr-review/
│   └── SKILL.md
└── failure-taxonomy/
    └── SKILL.md
```

9.5 Skill 템플릿

```
---
name: retrieval-eval
description: Run and interpret retrieval evaluation for Asperitas RAG changes.
---
```


Retrieval Eval Skill

Use this skill when a task changes chunking, metadata, retrieval scoring, ranking, or citation behavior.

Steps

1. Inspect changed files.
2. Run baseline eval if available.
3. Run updated eval.
4. Compare Top-K, citation accuracy, and failure cases.
5. Report before/after metrics.
6. If metrics regress, stop and recommend rollback or targeted fix.

Required report

- Query set used:
- Metrics before/after:
- Regressions:
- Improvements:
- Risk:

10. 새 채팅에서 성능향상/스킬추가/워크플로우 구축하는 법

본격 개발과 별개로 새 채팅을 열어 “성능향상”, “스킬추가”, “워크플로우/파이프라인 구축”을 따로 진행하는 것은 매우 좋은 전략이다. 개발 채팅과 운영개선 채팅을 분리하면 구현 context와 운영 system design context가 섞이지 않는다.

10.1 새 채팅을 여는 5 가지 상황

새 채팅 목적	언제 여는가	산출물
Performance Upgrade	반복적으로 Codex가 실수하거나 테스트가 느릴 때	AGENTS.md rule, skill, eval threshold 개선
Skill Addition	같은 체크리스트를 계속 붙여넣을 때	SKILL.md 파일
Workflow Pipeline	여러 사람이 같은 절차를 따라야 할 때	SOP, PR template, branch strategy
Open-source Scout	새 기술/라이브러리를 조사할 때	candidate matrix, ledger draft
Postmortem	작업이 실패하거나 지표가 악화했을 때	failure taxonomy, prevention rule

10.2 새 채팅 시작 프롬프트

We are improving the Asperitas Agent Factory workflow, not implementing code directly.

Context:

- Current repo: asperitas-agent
- Current MVP stage: [MVP-xxx]
- Recent failure or repeated friction: [describe]
- Current workflow asset: AGENTS.md / .agents/skills / PR template / eval script

Task:

Design an upgrade to our workflow or skill system so future Codex runs make fewer mistakes.

Constraints:

- Do not suggest vague best practices only.
- Output must be directly convertible into repo files.
- Include exact text for AGENTS.md or SKILL.md if needed.
- Include how to verify that the workflow improvement worked.

Report:

1. root cause
2. proposed durable rule
3. file/template to update
4. copy-paste block
5. verification method

10.3 성능향상은 모델 교체보다 루프 개선이 먼저다

- 프롬프트가 흐리면 더 좋은 모델도 scope creep을 일으킨다.
- AGENTS.md가 부정확하면 모든 Codex run이 같은 실수를 반복한다.
- 테스트가 없으면 AI가 만든 개선을 검증할 방법이 없다.
- eval이 없으면 검색/분류/생성 품질이 좋아졌는지 판단할 수 없다.
- 따라서 성능향상 순서는 Prompt -> AGENTS.md -> Skill -> Tests -> Eval -> Tooling -> Model 순서가 기본이다.

10.4 Workflow Upgrade Decision Tree

반복 실수 발생?

- └─ 요구사항이 매번 흐림 -> Prompt Template 개선
- └─ repo 규칙을 매번 설명함 -> AGENTS.md 개선
- └─ 절차가 길고 특정 작업에만 필요 -> Skill 생성
- └─ 검증을 자주 빼먹음 -> CI / hook / checklist 추가
- └─ 같은 종류의 버그 반복 -> regression test 추가
- └─ 결과 해석이 어려움 -> report format 개선
- └─ 작업이 너무 큼 -> MVP split / branch strategy 개선

11. Claude Code/Skills/Subagents 와 Codex 병행 전략

Codex 만으로도 충분히 개발을 진행할 수 있지만, Claude Code 를 병행하면 리뷰, 대규모 탐색, skill/hook/subagent 기반 워크플로우에 강점을 얻을 수 있다. 단, 두 도구가 같은 파일을 동시에 고치면 충돌과 책임소재 문제가 생기므로 역할을 분리해야 한다.

11.1 병행 운영 원칙

상황	Codex	Claude Code	주의점
MVP 구현	주 구현 엔진	리뷰 또는 대안 설계	동일 branch 동시 수정 금지
대규모 코드 탐색	가능	강함	탐색 결과를 문서화 후 Codex 프롬프트로 전달
Skill/Rules 설계	가능	강함	repo-specific 규칙은 AGENTS.md 와 충돌하지 않게
PR 리뷰	가능	강함	수정 없이 review only 로 실행
CI/GitHub automation	가능	Claude GitHub Actions 가능	권한/비용/보안 검토 필요

11.2 Claude Code 용 CLAUDE.md 템플릿

```
# CLAUDE.md
```

```
This repository follows the Asperitas Agent Factory workflow.
```

```
Core rules:
```

- Inspect before editing.
- Make the smallest safe change.
- Preserve backward compatibility.
- Add or update tests.
- Run verification commands and report results.
- Do not commit secrets or raw sensitive data.
- Do not modify the same branch concurrently with another agent.

```
When reviewing Codex output:
```

- Do not edit files unless asked.
- Identify blocking and non-blocking issues.
- Check tests, scope, backward compatibility, security, license, and compliance.

11.3 Subagent 역할 예시

Subagent	역할	출력
Codebase Explorer	관련 파일과 architecture map 탐색	file map, dependency notes
Test Engineer	테스트 누락과 regression fixture 설계	test plan, fixture cases
Security/License Reviewer	dependency, license, secret risk 검토	risk report, ledger update
RAG Evaluator	retrieval metrics 와 실패 유형 분석	eval table, failure taxonomy
Docs Engineer	README, PR summary, onboarding docs 갱신	docs patch

11.4 Codex 와 Claude 병행 금지 패턴

- 둘 다 같은 branch 에서 구현 patch 를 만들게 하지 않는다.
- Codex 가 만든 diff 를 Claude 가 review 없이 자동 수정하게 하지 않는다.
- Claude 가 제안한 대규모 리팩터링을 Codex 에게 한 번에 실행시키지 않는다.
- 두 도구의 상반된 조언을 검증 없이 섞지 않는다.
- 권한이 넓은 automation 을 secret 이 있는 repo 에 바로 연결하지 않는다.

12. 신규 개발자 7 일 온보딩 플랜

신규 개발자는 첫 주에 코드를 많이 치는 것보다 Asperitas 의 AI-assisted engineering 루프를 정확히 익혀야 한다. 아래 플랜은 개발자가 회사 방식으로 안전하게 합류하도록 설계한다.

Day	목표	해야 할 일	완료 기준
Day 1	Context 이해	README, AGENTS.md, 이 플레이북 1~6 장 읽기. repo clone. GitHub 권한 확인.	local setup 완료, 질문 5 개 정리
Day 2	개발 루프 관찰	최근 PR 2 개와 decision log 읽기. 테스트 명령 실행.	pytest 실행 로그 제출
Day 3	작은 문서 PR	docs typo 또는 README 개선을 feature branch 로 PR.	PR 생성, CI 통과
Day 4	테스트 PR	작은 unit test 또는 fixture 추가.	테스트 통과, review 반영
Day 5	Codex task 수행	ChatGPT 가 만든 프롬프트로 Codex 에 작은 변경 지시.	Codex report + diff review
Day 6	검증/회고	artifact verify/eval 이해. failure taxonomy 작성 연습.	MVP mini report 제출
Day 7	Skill 개선	반복 절차 하나를 SKILL.md 초안으로 작성.	skill PR 또는 proposal

12.1 개발자에게 첫날 설명할 한 문단

온보딩 메시지

우리 개발 방식은 AI 에게 대충 말하는 방식이 아니라, AI 를 강력한 실행 엔진으로 쓰되 GitHub, 테스트, eval, 리뷰로 통제하는 방식입니다. 당신의 역할은 코드를 많이 치는 사람이 아니라, 요구사항을 작게 쪼개고, AI 가 만든 변경을 검증하고, 반복되는 지식을 repo 의 스킬과 규칙으로 남기는 엔지니어입니다.

12.2 신규 개발자 금지 사항

- main branch 에 직접 push 금지.
- Codex/Claude 가 만든 코드를 diff 확인 없이 commit 금지.
- 테스트 없이 기능 완료 선언 금지.
- 라이선스 확인 없이 오픈소스 코드 복사 금지.
- 회사 내부 문서/데이터를 public repo 나 외부 서비스에 업로드 금지.
- 대형 리팩터링과 기능 추가를 하나의 PR 에 섞기 금지.

13. 복붙용 프롬프트 라이브러리

이 장은 실제 작업에서 바로 복붙하는 용도다. 필요한 값만 바꿔서 Codex 또는 ChatGPT에 넣는다.

13.1 MVP 구현 프롬프트

Use AGENTS.md and the relevant .agents/skills instructions.

Task:

Implement [MVP name / feature] for [agent/repo].

Context:

- Current repo: asperitas-agent
- Current MVP stage: [MVP-xxx]
- Target files: inspect first unless obvious
- Current behavior: [describe]
- Desired behavior: [describe]

Constraints:

- Do not delete files.
- Make the smallest safe change.
- Preserve backward compatibility.
- Add or update tests.
- Run pytest.
- Run artifact verification.
- Run retrieval eval if retrieval/chunking/scoring is affected.
- Update docs if behavior changes.
- Do not commit.

Report:

1. objective
2. files changed
3. tests run
4. eval metrics before/after
5. risks

13.2 버그 수정 프롬프트

Debug and fix the issue below.

Issue:

[paste error/log]

Requirements:

- Reproduce or explain the failure first.
- Add a regression test if possible.
- Make the smallest safe fix.
- Do not rewrite unrelated code.
- Run pytest for affected tests and full suite if practical.
- Report root cause, files changed, tests run, and residual risk.

13.3 PR 리뷰 프롬프트

Review only. Do not edit files.

Inspect the current diff and evaluate whether this PR is safe to merge.

Check:

- scope match
- correctness
- tests
- backward compatibility
- security/privacy
- license/IP
- docs
- maintainability

Return blocking issues, non-blocking suggestions, and merge recommendation.

13.4 오픈소스 후보 조사 프롬프트

Research open-source projects relevant to [topic]. Do not modify files.

For each candidate, report URL, license, maintenance signal, architecture value, security risk, adoption risk, and recommended action.

Then propose a safe adaptation plan that avoids uncontrolled copy-paste.

13.5 Skill 생성 프롬프트

Create a reusable skill for the repeated workflow below.

Workflow pain:

[describe repeated task or repeated error]

Output:

- SKILL.md content
- where to place it under .agents/skills/
- when the skill should trigger
- required checklist
- verification method
- example Codex prompt using the skill

13.6 Failure Postmortem 프롬프트

Analyze this failed Codex/agent run.

Inputs:

- original prompt:
- Codex report:
- failing logs:
- git diff summary:

Return:

1. root cause
2. failure taxonomy label
3. what should have stopped earlier
4. durable AGENTS.md rule
5. skill/checklist update
6. regression test suggestion

14. 운영 리스크와 금지 사항

리스크	왜 위험한가	방지책
AI 과신	테스트 없이 잘못된 로직이 main 에 들어감	검증 게이트 필수화
Scope creep	작은 작업이 대규모 리팩터링으로 변질	Preflight와 stop rule
오픈소스 라이선스 오염	상업적 사용/IP 문제 가능	License gate, ledger
비밀정보 유출	API key, 내부자료 유출	secret scan, .gitignore, private repo
데이터 품질 저하	RAG agent 가 잘못된 근거를 인용	source hierarchy, citation eval
평가 부재	품질 개선 여부 판단 불가	golden set, eval script
팀 지식 분산	개인 채팅에만 노하우 존재	AGENTS.md, Skills, decision logs
동시 AI 수정 충돌	Codex/Claude 가 같은 파일을 서로 덮어씀	branch isolation, review-only mode

14.1 Red Flag: 즉시 중단해야 하는 상황

- Codex 가 요청하지 않은 파일을 대량 수정했다.
- 테스트를 삭제하거나 약화했다.
- 기존 PASS/FAIL 기준을 바꾸면서 그 이유를 설명하지 못한다.
- 새 dependency 를 추가했지만 license/security 검토가 없다.
- 외부 네트워크 호출, API key, raw internal data 가 코드나 로그에 들어갔다.
- retrieval eval 이 약화했는데 “대체로 괜찮다”고 보고한다.
- 한 PR 에서 feature, refactor, formatting, docs 가 모두 섞였다.

14.2 Biosafety/Compliance 관점

Asperitas 의 생물학/합성생물학 관련 에이전트는 일반 소프트웨어보다 엄격한 기준을 가져야 한다. 초기 MVP 단계에서는 실험 조건, 유전자 설계, 병원성/독성/규제 회피 조건을 자동 생성하거나 실행하도록 만들지 않는다. 내부 문서 검색과 근거 정리, 체크리스트 보조, 규제 문서 요약처럼 보수적인 범위에서 시작한다.

허용 초기 범위	금지/승인 필요 범위
논문/규제/내부 문서 검색 및 근거 인용	실험 조건 자동 확정, 위험 생물체 조작 지시
SOP 체크리스트와 audit log 보조	규제 회피, biosafety 절차 생략 제안
기존 공개 지식 요약과 출처 정리	검증되지 않은 wet-lab protocol 생성
리스크 라벨링과 사람 검토 요청	자동 실행/로봇 제어 연결

15. KPI와 성능향상 루프

Agent Factory의 성능은 “AI가 답을 잘한다”가 아니라 개발 생산성과 품질 지표로 측정해야 한다. 아래 KPI를 월 단위로 본다.

KPI	정의	목표 방향
Prompt-to-PR cycle time	초안부터 PR 생성까지 걸린 시간	감소
First-pass test rate	Codex 구현 후 첫 pytest 통과 비율	증가
Scope creep rate	요청 범위 밖 파일 수정 비율	감소
Regression rate	merge 후 깨지는 테스트/평가 비율	감소
Eval improvement rate	retrieval/classification metric 개선 빈도	증가
Skill reuse count	Skill/AGENTS.md 규칙 재사용 횟수	증가
Open-source adoption quality	ledger가 있는 dependency/코드 채택 비율	증가
Postmortem closure rate	반복 오류가 durable rule/test로 닫힌 비율	증가

15.1 월간 Agent Factory Review

Monthly Agent Factory Review

1. Completed MVPs:
2. PR count / merge count:
3. Average cycle time:
4. Tests added:
5. Eval metrics improved/regressed:
6. Top 5 repeated failures:
7. AGENTS.md updates:
8. Skills added/updated:
9. Open-source candidates adopted/rejected:
10. Next month focus:

15.2 개선 우선순위 공식

개선은 가장 화려한 자동화부터 하지 않는다. 반복 빈도와 위험도를 곱해서 우선순위를 정한다.

개선 후보	빈도	위험도	우선순위
Codex가 테스트를 빼먹음	높음	높음	즉시 skill/AGENTS/CI 개선
commit message가 부정확함	중간	낮음	PR template 개선
retrieval eval 해석이 어려움	중간	높음	eval report format 개선
오픈소스 license 확인 누락	낮음	매우 높음	ledger와 license gate 즉시 도입

16. Appendix: 파일 템플릿, 체크리스트, 공식 출처

16.1 MVP Decision Log 템플릿

```
# MVP-xxx Decision Log

Date:
Owner:
Repo:
Branch:
Commit/PR:

## Objective

## Context

## Changes

## Verification
- pytest:
- artifact verification:
- eval before/after:

## Decision
Accept / Reject / Revise

## Risks

## Follow-ups

## Durable lessons
- AGENTS.md update needed?
- Skill update needed?
- Regression test needed?
```

16.2 CODEOWNERS 예시

```
# CODEOWNERS
/src/*                @engineering-lead
/src/*retrieval*      @ai-manager @retrieval-owner
/src/*ingestion*      @data-owner
/scripts/run_retrieval_eval.py @ai-manager
/.agents/skills/*     @ai-manager
/docs/*               @ops-owner
```

16.3 GitHub Actions pytest 예시

```
name: tests

on:
  pull_request:
  push:
    branches: [ main ]

jobs:
  pytest:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: python -m pip install -U pip
      - run: pip install -e .[dev]
      - run: python -m pytest
```

16.4 공식 출처

아래 출처는 2026-06-15 기준 v2.0 작성에 참고한 공개 공식 문서다. 제품 기능과 가격/권한은 변경될 수 있으므로 실제 도입 전 최신 문서를 다시 확인한다.

ID	Source	URL
S1	OpenAI Codex Best Practices	https://developers.openai.com/codex/learn/best-practices
S2	OpenAI Codex CLI	https://developers.openai.com/codex/cli
S3	OpenAI Codex Prompting Guide / AGENTS.md	https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide
S4	OpenAI Codex Subagents	https://developers.openai.com/codex/subagents

S5	OpenAI Codex with Agents SDK	https://developers.openai.com/codex/guides/agents-sdk
S6	Anthropic Claude Code Overview	https://docs.anthropic.com/en/docs/claude-code/overview
S7	Anthropic Claude Code Skills	https://docs.anthropic.com/en/docs/claude-code/skills
S8	Anthropic Claude Code Hooks	https://docs.anthropic.com/en/docs/claude-code/hooks-guide
S9	Anthropic Claude Code Subagents	https://docs.anthropic.com/en/docs/claude-code/sub-agents
S10	GitHub Protected Branches	https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
S11	GitHub Managing Branch Protection Rules	https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/managing-a-branch-protection-rule
S12	GitHub About Pull Requests	https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

16.5 Final Operating Rule

최종 원칙

Asperitas의 독자적 에이전트 개발은 “AI에게 맡기기”가 아니라 “AI를 이용해 검증 가능한 엔지니어링 루프를 고속화하기”다. 모든 프롬프트는 다음 커밋의 품질을 결정하고, 모든 커밋은 다음 에이전트의 학습 데이터가 된다.